

Document Metadata

- **Project Name:** BridgeAI - Multimodal Logistics Platform
- **Academic Context:** PFE (Projet de Fin d'Études) - Data Analytics & Web Engineering

2. Product Overview

The BridgeAI backend is a robust API layer that handles trade compliance, logistics data pipelines, AI-driven delay predictions, and multimodal route optimization for freight moving between North Africa (Morocco) and Europe.

3. Technical Stack

- **Framework:** FastAPI (Python)
- **Database:** Supabase (PostgreSQL)
- **Graph Engine:** NetworkX ([MultiDiGraph](#))
- **Machine Learning:** Scikit-Learn / XGBoost (Delay Prediction)
- **Data Processing:** Pandas, NumPy

4. Database Schema (Supabase)

Table: [optimisation_data](#) (The Graph Edges)

- [id](#) (UUID, Primary Key)
- [origin_node](#) (String, e.g., "Casablanca", "Tanger Med")
- [dest_node](#) (String)
- [transport_mode](#) (String: Air, Sea, Rail, Road, Transfer)
- [carrier_name](#) (String)
- [distance_km](#) (Float)
- [base_time_hours](#) (Float)
- [base_cost_usd](#) (Float)
- [co2_emissions_kg](#) (Float)
- [carrier_reliability_score](#) (Float, 0.0 to 1.0)
- [risk_factor](#) (Float, 0.0 to 1.0)

Table: [historical_shipments](#) (For ML Training)

- [shipment_id](#) (UUID)
- [route_id](#) (UUID, Foreign Key)
- [weather_condition](#) (String)
- [port_congestion_index](#) (Float)
- [actual_delay_hours](#) (Float - Target Variable)

5. Core Modules & Algorithms

Module A: The AI Delay Predictor

- **Logic:** An XGBoost or Logistic Regression model trained on historical Hub-and-Spoke routing datasets to predict `actual_delay_hours` based on real-time weather and port congestion.
- **Function:** Injects a dynamic time penalty into the routing engine if high risk is detected.

Module B: The Graph Routing Engine

- **Logic:** Constructs a `MultiDiGraph` on server startup.
- **Mathematics:** 1. $\$Penalty = 1.0 - Reliability_Score\$$
2. Applies Min-Max Normalization to Time, Cost, CO2, and Penalty.
3. Calculates composite edge weight based on user presets (Fastest, Cheapest, Eco, Premium, Balanced).
- **Execution:** Runs Dijkstra's algorithm using the composite weight.

6. API Specifications

POST /api/route/optimize

Request Payload:

JSON

```
{
  "origin": "Casablanca",
  "destination": "Paris",
  "preset": "eco"
}
```

Response (200 OK): Returns `total_cost`, `total_time`, `total_co2`, and an ordered array `path` of steps containing the exact mode and carrier for each hop.

GET /api/nodes

Response (200 OK): Returns a unique list of all available logistics hubs for frontend dropdowns.

POST /api/predict/delay

Request Payload: `{ "origin": "Tanger Med", "destination": "Algeciras", "weather": "storm" }`

Response (200 OK): `{ "predicted_delay_hours": 12.5, "confidence_score": 0.88 }`

7. Automated Testing Criteria (TestSprite / PyTest)

- **TC-BE-001 (Unit - Graph):** The application must successfully build an `nx.MultiDiGraph` in memory upon FastAPI startup without throwing connection errors to Supabase.
- **TC-BE-002 (E2E - Optimization):** Sending `preset="cheapest"` must mathematically yield the path with the absolute minimum `base_cost_usd` sum, validating the Dijkstra implementation.

- **TC-BE-003 (Error Handling):** Requesting an optimization between two unconnected nodes (e.g., isolated graphs) must return a **400 Bad Request** or **404 Not Found** with a specific JSON error message, not a **500 Internal Server Error**.
- **TC-BE-004 (ML Integration):** The delay prediction endpoint must return a float value ≥ 0 for any valid string input of weather and nodes.